

RECURSION

DSA Notes - from the first idea to Linked Lists and Trees

The simplest way to think about recursion: solve a smaller version of the same problem, then let the function calls return.

What this chapter covers

We will build recursion from scratch: function calls, base cases, recursive cases, call stack, tracing, arrays, strings, linked lists, and trees. The goal is to understand why recursive code works instead of memorising templates.

A recursive function has two parts:

1. Base case -> when to stop
2. Recursive case -> make the problem smaller

Rule to remember: every recursive call must move toward a base case.

1. FUNCTION CALLS

Before recursion, understand what happens when one function calls another.

```
void A() {  
    cout << "A";  
    B();  
}  
  
void B() {  
    cout << "B";  
}  
  
int main() {  
    A();  
}
```

Execution:

```
main()  
  |  
  v  
A()  
  |  
  v  
B()  
  |  
  v  
return
```

Then B finishes -> A finishes -> main continues

When a function calls another function, the current function waits for that call to finish. Recursion uses exactly the same mechanism - except the function calls itself.

2. FIRST RECURSIVE FUNCTION

```
void fun() {  
    cout << "Hello\n";  
    fun();  
}
```

This never stops. There is no base case.

```
fun()  
-> fun()  
    -> fun()  
        -> fun()  
            -> ...
```

So a recursive function needs a stopping condition.

```
void fun(int n) {  
    if(n == 0)  
        return;  
  
    cout << n << " ";  
    fun(n - 1);  
}
```

For fun(5):

```
fun(5)
fun(4)
fun(3)
fun(2)
fun(1)
fun(0) -> return
```

Base case = the smallest problem that can be answered directly.

3. BASE CASE AND RECURSIVE CASE

Most recursion problems can be read in two pieces.

```
void solve(int n) {

    // BASE CASE
    if(n == 0)
        return;

    // WORK

    // RECURSIVE CASE
    solve(n - 1);
}
```

The recursive call should make the input smaller or move closer to a stopping point.

Bad recursion:

```
void solve(int n) {
    if(n == 0) return;
    solve(n);        // n never changes
}
```

Good recursion:

```
void solve(int n) {
    if(n == 0) return;
    solve(n - 1);
}
```

4. PRINTING 1 TO N AND N TO 1

The position of the print statement changes the answer.

1 TO N:

```
void print(int n) {
    if(n == 0) return;

    print(n - 1);
    cout << n << " ";
}
```

```
print(5)
-> print(4)
    -> print(3)
        -> print(2)
            -> print(1)
                -> print(0)
```

Returning:
1 2 3 4 5

N TO 1:

```
void print(int n) {  
    if(n == 0) return;  
  
    cout << n << " ";  
    print(n - 1);  
}
```

Going down:

5 -> 4 -> 3 -> 2 -> 1

Answer:

5 4 3 2 1

Before the recursive call = work happens while going down. After the recursive call = work happens while coming back up.

5. CALL STACK

Every function call gets its own stack frame. Recursive calls build a stack of the same function.

```
void fun(int n) {  
    if(n == 0) return;  
    cout << n << " ";  
    fun(n - 1);  
}
```

fun(3)

While fun(1) is running, the stack looks roughly like this:

```
TOP  
+-----+  
| fun(1) |  
+-----+  
| fun(2) |  
+-----+  
| fun(3) |  
+-----+  
| main() |  
+-----+
```

fun(1) finishes first. Then fun(2), then fun(3). This is why recursion naturally behaves like a stack: last call in, first call out.

6. TRACE RECURSION ON PAPER

For a new recursive problem, write each call on a new line and mark where the function returns.

```
int sum(int n) {  
    if(n == 0) return 0;  
    return n + sum(n - 1);  
}
```

sum(4)

```
sum(4) = 4 + sum(3)
sum(3) = 3 + sum(2)
sum(2) = 2 + sum(1)
sum(1) = 1 + sum(0)
sum(0) = 0
```

Returning:
sum(1) = 1
sum(2) = 3
sum(3) = 6
sum(4) = 10

Do not try to mentally execute everything at once. Separate the two phases: going down and coming back up.

7. RECURSION ON ARRAYS

The common idea is to keep an index. Each call handles one position and moves to the next.

PRINT AN ARRAY:

```
void print(int a[], int i, int n) {
    if(i == n) return;

    cout << a[i] << " ";
    print(a, i + 1, n);
}
```

For:

```
a = [10, 20, 30, 40]
print(a, 0, 4)
```

```
i = 0 -> print 10
i = 1 -> print 20
i = 2 -> print 30
i = 3 -> print 40
i = 4 -> stop
```

SUM OF ARRAY:

```
int sum(int a[], int i, int n) {
    if(i == n) return 0;

    return a[i] + sum(a, i + 1, n);
}
```

REVERSE AN ARRAY:

```
void reverse(int a[], int l, int r) {
    if(l >= r) return;

    swap(a[l], a[r]);
    reverse(a, l + 1, r - 1);
}
```

Here the problem gets smaller from both sides.

8. RECURSION ON STRINGS

Strings work the same way: keep an index or two pointers and reduce the remaining part.

PALINDROME CHECK:

```
bool isPal(string &s, int l, int r) {
    if(l >= r) return true;

    if(s[l] != s[r]) return false;

    return isPal(s, l + 1, r - 1);
}
```

"level"

```
l       r
|       |
v       v
l e v e l
```

then:
e v e

then:
v

stop -> true

Array/string recursion usually means: choose a position, process it, then recurse on the remaining range.

9. RECURSION WITH LINKED LIST

A linked list already gives us a natural next step: node -> next.

10 -> 20 -> 30 -> NULL

Recursive traversal:

```
void print(Node* root) {
    if(root == NULL)
        return;

    cout << root->data << " ";
    print(root->next);
}
```

```
print(10)
-> print(20)
    -> print(30)
        -> print(NULL)
```

Output:
10 20 30

Reverse-order printing is a simple example of work done while returning:

```
void printReverse(Node* root) {
    if(root == NULL)
        return;

    printReverse(root->next);
    cout << root->data << " ";
}
```

This prints:

30 20 10

The linked list is useful because each node points to exactly one next node. The recursive call follows that pointer.

10. FROM LINKED LIST TO TREE

This is the important connection.

Linked List	Binary Tree
10 -> 20 -> 30	10 / 20 30
Node has one link next	Node has two links left + right

Linked List recursion:

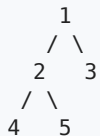
```
solve(root->next);
```

Tree recursion:

```
solve(root->left);  
solve(root->right);
```

In a linked list, from one node there is normally one next node. In a binary tree, from one node there may be a left child and a right child. That is why tree recursion branches.

11. RECURSION ON TREES



Basic DFS pattern:

```
void dfs(Node* root) {  
    if(root == NULL)  
        return;  
  
    dfs(root->left);  
    dfs(root->right);  
}
```

PREORDER = Root, Left, Right

```
void preorder(Node* root) {  
    if(root == NULL) return;  
  
    cout << root->data << " ";  
    preorder(root->left);  
    preorder(root->right);  
}
```

Output:

```
1 2 4 5 3
```

INORDER = Left, Root, Right

```
void inorder(Node* root) {
    if(root == NULL) return;

    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

Output:

4 2 5 1 3

POSTORDER = Left, Right, Root

```
void postorder(Node* root) {
    if(root == NULL) return;

    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

Output:

4 5 2 3 1

The three traversals are the same recursion. Only the position of the work changes.

12. COMMON RECURSION PATTERNS

Pattern 1 - Move one step:

```
solve(i + 1);
```

Used for arrays, strings, linked lists.

Pattern 2 - Shrink from both sides:

```
solve(l + 1, r - 1);
```

Used for palindrome checks and divide-from-ends problems.

Pattern 3 - Two branches:

```
solve(root->left);
solve(root->right);
```

Used heavily in binary trees.

Pattern 4 - Choose / do not choose:

```
solve(i + 1);
solve(i + 1);
```

This is the basis of many subsequence, subset, and backtracking problems. The two calls represent two different choices, so the exact arguments normally differ.

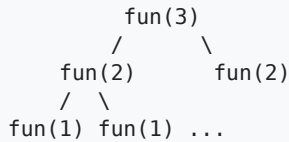
13. MULTIPLE RECURSIVE CALLS

Consider:

```
void fun(int n) {
    if(n == 0) return;

    fun(n - 1);
    fun(n - 1);
}
```


For $n = 3$:



One call creates another call, so the number of calls grows quickly. This is why some recursive algorithms are exponential.

Do not confuse recursion with slow code. Recursion itself is not the problem; the number of recursive calls and the work done at each call determine the complexity.

14. TAIL RECURSION VS WORK AFTER RECURSION

Two functions can look almost identical but behave differently.

```
void f(int n) {
    if(n == 0) return;
    cout << n << " ";
    f(n - 1);
}
```

The print happens before the recursive call, so values appear while going down.

```
void g(int n) {
    if(n == 0) return;
    g(n - 1);
    cout << n << " ";
}
```

The print happens after the recursive call, so values appear while returning.

This small change is responsible for many common recursion questions.

15. WHEN RECURSION FAILS

A recursive solution usually has one of these mistakes:

1. No base case
2. Recursive call never reaches the base case
3. Wrong state is passed to the next call
4. Too many recursive calls
5. Recomputing the same subproblem again and again

Example - wrong movement:

```
solve(i);
```

Example - correct movement:

```
solve(i + 1);
```

For large inputs, deep recursion can also overflow the call stack. The exact safe depth depends on the language and environment.

16. HOW TO WRITE A RECURSIVE SOLUTION

When facing a new recursion problem, ask these questions in order:

1. What is the smallest case?
2. What should happen there?
3. What is my current state?
4. How do I make the problem smaller?
5. What does the recursive call return?
6. Where should my current work happen?

Example: sum from 1 to n

```
int sum(int n) {  
    if(n == 0) return 0;  
    return n + sum(n - 1);  
}
```

Read it as:

sum(n) = n + answer for the smaller problem (n - 1).

17. RECURSION CHEAT SHEET

```
LINKED LIST  
    current -> next  
    solve(current->next)  
  
ARRAY  
    index -> index + 1  
    solve(i + 1)  
  
STRING / TWO POINTERS  
    left++, right--  
    solve(l + 1, r - 1)  
  
BINARY TREE  
    left child + right child  
    solve(root->left)  
    solve(root->right)
```

And the basic skeleton:

```
void solve(State state) {  
  
    if(base_case)  
        return;  
  
    // current work  
  
    solve(smaller_state);  
}
```

If you can explain what the current call means, what the base case means, and why the next call is smaller, you understand the recursion.

RECURSION -> LINKED LIST -> TREE

The same idea keeps getting extended: follow one next pointer in a linked list; follow two child pointers in a binary tree. The syntax changes very little. The branching changes everything.